Independent University, Bangladesh
CSE 211 (Algorithms)
Section: 2

Project Title: Comprehensive Review of the A* Algorithm: Theory, Implementation, and Applications

S.M.Masruk Uddin

ID: 1820855
Department of Physical Sciences

TOUSHIF HOSSAIN

ID:1920404
Department of Computer Science and Engineering

Suhaila

ID: 1910496
Department of Computer Science and Engineering

Tasnia Tabassum Oishee

ID: 1910040
Department of Computer Science and Engineering

Azwad Fawad Hasan

ID:2020222
Department of Computer Science and Engineering

Summer 2021

1 Introduction of the Paradigm of the Algorithm (Inventor, History, etc.)

"*You don't want to take a long time churning through the roads unnecessary...But you can see this is the kind of thing we are doing. We are trying to find a nice elegant way of getting from A to B.*"

– Dr Michael P. Pound (University of Nottingham), While discussing A* Algorithm on Computerphile.

A* Algorithms traverse the graph in search of the optimal path between a starting point and an end point. This Algorithm is direct offshoot of the Shakey project. The project was started in 1966 by Artificial Intelligence Center of Stanford Research Institute[3]. The first general-purpose mobile robot - Shakey the Robot was able to reason about its own actions, other robots at that time would have to be given procedure on each individual step of completing a big task, whereas Shakey could analyze commands and break them down into basic parts by itself. The creation of Shakey the robot had a tremendous impact on the fields of robotics and artificial intelligence. The development of A* algorithm was the noticeable result of this experiment, this algorithm is now widely used in pathfinding and graph traversal, the procedure of plotting productive traversal and path between points.The two paradigm of this A* Algorithm is Dynamic Programming and Greedy Algorithm.

1.1 Dynamic programming

It is used for finding the shortest path in a graph using optimal substructure. This method was created by Richard Bellman in 1940s and developed till 1950s.

1.2 Greedy algorithm

It is used for finding an optimized result from feasible solutions. It was invented by Edsger W. Dijkstra when he desired to calculate the minimum spanning tree. During 1950s this algorithm started to shine when Prim and Kruskal also achieved the optimization techniques for minimizing the costs of graphs during that decade. American researchers suggested a recursive strategy for solving greedy problems during 1970s. Finally in 2005, the NIST records inscribed the greedy paradigm as a separate optimization strategy.

2 Introduction to the A* Algorithm

A* search is a computer algorithm which is widely used in shortest path-finding and graph traversal. It is an advanced best-first search algorithm that aims to inquiries through all the available paths with the least cost (shortest path); thus, A* is said to be optimal and a complete Algorithm. In 1968, A* search algorithm was first developed to mix heuristic approaches like Greedy Best-First-Search and formal approaches like Dijkstra's Algorithm. Due to this reason, it is often used in many fields of computer science, such as games and web-based maps that use this algorithm to find the shortest path very efficiently.

Each node carries a certain amount of weight, and the A* the search algorithm first starts from a specific starting node and aims to find the shortest path towards its destination. The equation below is the basic theoretical representation of how the algorithm works.

$$f(n) = g(n) + h(n)$$

Here,
$-n$ is the next node on the path.
$-g(n)$ is the cost of the path from the beginning node to n.
$-h(n)$ is a heuristic function that estimates the value of the most cost-effective path from n to the goal.

A* has to determine which of its paths to extend, and it completes its task based on the path's cost and an estimation of the cost required to extend the path to the goal at each repetition of the loop. So A* terminates from the loop mainly due to two factors: when the path it chooses to extend is a path that leads from start to goal or if there are no paths available to be extended. On the other hand, The heuristic function is problem-specific. It accurately provides the actual cost to get to the goal; therefore, the algorithm for a fact will return a least-cost path from start to goal. Meanwhile, the nodes with the lowest $f(n)$ values are the path chosen to be taken from start to finish.

3 Flowchart and Pseudocode

Flowchart [6] of the A* algorithm:

The process below starts from two lists, OPEN and CLOSED. Where CLOSED contains those visited nodes and OPEN contains those nodes that have been evaluated by the heuristic function but have not been expanded into successors yet.

Step-01:

Define a list OPEN.

It consists the start node S.

Step-02:

If the list is empty, exit.

Step-03:

Remove node n with the smallest value of $f(n)$ from OPEN and move it to list CLOSED.

If node n is a goal state, return success and exit.

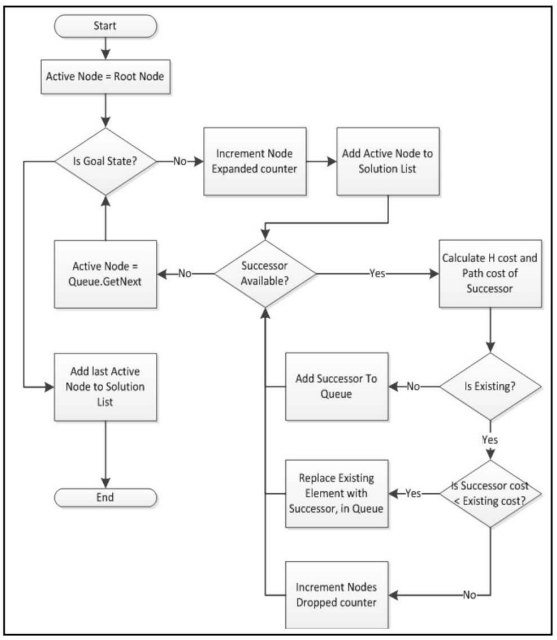
Step-04:

Expand node n.

Step-05:

If any successor to n is the goal node, return success and the solution by tracing the path from goal node to S.

Otherwise, go to Step-06.



Step-06:

For each successor node,Apply $f(n) = g(n) + h(n)$.

If the node has not been in either list, add it to OPEN.

Step-07:

Go back to Step-02

3.1 Pseudocode

```
//After initializing OPEN and CLOSES list

While(Open List is not empty)
{

    a) Obtain the node with the least f on the Open List
and name it    q
    b) Remove    q    from the Open List
    c) Compute successors from  q s . Fix their parents to    q
    d) For (a specific successor)
    {

        i) Terminate the search only if the successor is the required goal.
           successor.g = q.g + the total distance between the successor and q.
           successor.h =  the total distance from the goal to the successor
               /*(This can be found using three methods-Manhattan, Diagonal
               and Euclidean Heuristics)*/

               //f(n)=h(n)+g(n)
               successor.f =  successor.h + successor.g


        ii) Leave out the successor, if a specific node is in the Closed List
           (which has a lower f than the  successor) and if it also has the same
           position as the successor. Else, add the specific node to the Open List.

        iii) Leave out the successor, if a specific node with the same position
        as the successor is in the Open List, which has a lower f than successor.

        //Terminate For Loop
    }
    e) Insert q on the Closed List

//Terminate While loop
}
```

4 Implementation

The algorithm's implementation is solely determined by the problem at hand. The problem might demand various data structure or dynamic programming. Here is an example of 9×10 grid type graph. And the code[2] intends to find the optimal path between two points.

```
#include <bits/stdc++.h>
using namespace std;

#define ROW 9
#define COL 10

typedef pair<int, int> Pair;

typedef pair<double, pair<int, int> > pPair;

struct cell {

        int parent_i, parent_j;

        double f, g, h;
};

bool isValid(int row, int col)
{

        return (row >= 0) && (row < ROW) && (col >= 0)
                && (col < COL);
}

bool isUnBlocked(int grid[][COL], int row, int col)
{

        if (grid[row][col] == 1)
                return (true);
        else
                return (false);
}

bool isDestination(int row, int col, Pair dest)
{
        if (row == dest.first && col == dest.second)
                return (true);
        else
                return (false);
}

double calculateHValue(int row, int col, Pair dest)
{
        return ((double)sqrt(
                (row - dest.first) * (row - dest.first)
                + (col - dest.second) * (col - dest.second)));
}

void tracePath(cell cellDetails[][COL], Pair dest)
{
        cout <<"\n\n Path is ";
        int row = dest.first;
        int col = dest.second;

        stack<Pair> Path;

        while (!(cellDetails[row][col].parent_i == row
```

```
                        && cellDetails[row][col].parent_j == col)) {
                Path.push(make_pair(row, col));
                int temp_row = cellDetails[row][col].parent_i;
                int temp_col = cellDetails[row][col].parent_j;
                row = temp_row;
                col = temp_col;
        }

        Path.push(make_pair(row, col));
        while (!Path.empty()) {
                pair<int, int> p = Path.top();
                Path.pop();
                cout << "-> (" << p.first << "," << p.second <<") " ;
        }

        return;
}

void aStarSearch(int grid[][COL], Pair src, Pair dest)
{

        if (isValid(src.first, src.second) == false) {
                cout<<"Source is invalid\n";
                return;
        }


        if (isValid(dest.first, dest.second) == false) {
                cout <<"Destination is invalid\n";
                return;
        }


        if (isUnBlocked(grid, src.first, src.second) == false
                || isUnBlocked(grid, dest.first, dest.second)
                        == false) {
                cout <<"Source or   destination is blocked\n";
                return;
        }


        if (isDestination(src.first, src.second, dest)
                == true) {
                cout <<"We are already at   destination\n";
                return;
        }

    bool closedList[ROW][COL];
        memset(closedList, false, sizeof(closedList));

    cell cellDetails[ROW][COL];

        int i, j;

    for (i = 0; i < ROW; i++) {
                for (j = 0; j < COL; j++) {
                        cellDetails[i][j].f = FLT_MAX;
                        cellDetails[i][j].g = FLT_MAX;
```

```
                    cellDetails[i][j].h = FLT_MAX;
                    cellDetails[i][j].parent_i = -1;
                    cellDetails[i][j].parent_j = -1;
            }
    }

i = src.first, j = src.second;
    cellDetails[i][j].f = 0.0;
    cellDetails[i][j].g = 0.0;
    cellDetails[i][j].h = 0.0;
    cellDetails[i][j].parent_i = i;
    cellDetails[i][j].parent_j = j;

set<pPair> openList;

openList.insert(make_pair(0.0, make_pair(i, j)));

bool foundDest = false;


while (!openList.empty()) {
            pPair p = *openList.begin();


            openList.erase(openList.begin());


            i = p.second.first;
            j = p.second.second;
            closedList[i][j] = true;


            double gNew, hNew, fNew;


            if (isValid(i - 1, j) == true) {

                    if (isDestination(i - 1, j, dest) == true) {

                            cellDetails[i - 1][j].parent_i = i;
                            cellDetails[i - 1][j].parent_j = j;
                            cout <<"␣␣destination␣cell␣is␣found\n";
                            tracePath(cellDetails, dest);
                            foundDest = true;
                            return;
                    }

                    else if (closedList[i - 1][j] == false
                                    && isUnBlocked(grid, i - 1, j)
                                                    == true) {
                            gNew = cellDetails[i][j].g + 1.0;
                            hNew = calculateHValue(i - 1, j, dest);
                            fNew = gNew + hNew;


                            if (cellDetails[i - 1][j].f == FLT_MAX
                                    || cellDetails[i - 1][j].f > fNew) {
                                    openList.insert(make_pair(
```

```
                                fNew, make_pair(i - 1, j)));

                        cellDetails[i - 1][j].f = fNew;
                        cellDetails[i - 1][j].g = gNew;
                        cellDetails[i - 1][j].h = hNew;
                        cellDetails[i - 1][j].parent_i = i;
                        cellDetails[i - 1][j].parent_j = j;
                }
        }
}



if (isValid(i + 1, j) == true) {

        if (isDestination(i + 1, j, dest) == true) {

                cellDetails[i + 1][j].parent_i = i;
                cellDetails[i + 1][j].parent_j = j;
                cout <<"\u\u destination cell is found\n";
                tracePath(cellDetails, dest);
                foundDest = true;
                return;
        }

        else if (closedList[i + 1][j] == false
                        && isUnBlocked(grid, i + 1, j)
                                        == true) {
                gNew = cellDetails[i][j].g + 1.0;
                hNew = calculateHValue(i + 1, j, dest);
                fNew = gNew + hNew;


                if (cellDetails[i + 1][j].f == FLT_MAX
                        || cellDetails[i + 1][j].f > fNew) {
                        openList.insert(make_pair(
                                fNew, make_pair(i + 1, j)));

                        cellDetails[i + 1][j].f = fNew;
                        cellDetails[i + 1][j].g = gNew;
                        cellDetails[i + 1][j].h = hNew;
                        cellDetails[i + 1][j].parent_i = i;
                        cellDetails[i + 1][j].parent_j = j;
                }
        }
}


if (isValid(i, j + 1) == true) {

        if (isDestination(i, j + 1, dest) == true) {

                cellDetails[i][j + 1].parent_i = i;
                cellDetails[i][j + 1].parent_j = j;
                cout<<"\u\u destination cell is found\n";
                tracePath(cellDetails, dest);
```

```
                        foundDest = true;
                        return;
                }

                else if (closedList[i][j + 1] == false
                                && isUnBlocked(grid, i, j + 1)
                                                == true) {
                        gNew = cellDetails[i][j].g + 1.0;
                        hNew = calculateHValue(i, j + 1, dest);
                        fNew = gNew + hNew;


                        if (cellDetails[i][j + 1].f == FLT_MAX
                                || cellDetails[i][j + 1].f > fNew) {
                                openList.insert(make_pair(
                                        fNew, make_pair(i, j + 1)));


                                cellDetails[i][j + 1].f = fNew;
                                cellDetails[i][j + 1].g = gNew;
                                cellDetails[i][j + 1].h = hNew;
                                cellDetails[i][j + 1].parent_i = i;
                                cellDetails[i][j + 1].parent_j = j;
                }
                }
}

if (isValid(i, j - 1) == true) {

                if (isDestination(i, j - 1, dest) == true) {

                        cellDetails[i][j - 1].parent_i = i;
                        cellDetails[i][j - 1].parent_j = j;
                        cout<<"\u\u destination\u cell\u is\u found\n";
                        tracePath(cellDetails, dest);
                        foundDest = true;
                        return;
                }

    else if (closedList[i][j - 1] == false
                                && isUnBlocked(grid, i, j - 1)
                                                == true) {
                        gNew = cellDetails[i][j].g + 1.0;
                        hNew = calculateHValue(i, j - 1, dest);
                        fNew = gNew + hNew;
        if (cellDetails[i][j - 1].f == FLT_MAX
                                || cellDetails[i][j - 1].f > fNew) {
                                openList.insert(make_pair(
                                        fNew, make_pair(i, j - 1)));


                                cellDetails[i][j - 1].f = fNew;
                                cellDetails[i][j - 1].g = gNew;
                                cellDetails[i][j - 1].h = hNew;
                                cellDetails[i][j - 1].parent_i = i;
                                cellDetails[i][j - 1].parent_j = j;
                        }
                }
```

```
        }

if (isValid(i - 1, j + 1) == true) {

                if (isDestination(i - 1, j + 1, dest) == true) {

                        cellDetails[i - 1][j + 1].parent_i = i;
                        cellDetails[i - 1][j + 1].parent_j = j;
                        cout <<"\u\u destination cell is found\n";
                        tracePath(cellDetails, dest);
                        foundDest = true;
                        return;
                }

    else if (closedList[i - 1][j + 1] == false
                                && isUnBlocked(grid, i - 1, j + 1)
                                                == true) {
                        gNew = cellDetails[i][j].g + 1.414;
                        hNew = calculateHValue(i - 1, j + 1, dest);
                        fNew = gNew + hNew;


                        if (cellDetails[i - 1][j + 1].f == FLT_MAX
                                || cellDetails[i - 1][j + 1].f > fNew) {
                                openList.insert(make_pair(
                                        fNew, make_pair(i - 1, j + 1)));


                                cellDetails[i - 1][j + 1].f = fNew;
                                cellDetails[i - 1][j + 1].g = gNew;
                                cellDetails[i - 1][j + 1].h = hNew;
                                cellDetails[i - 1][j + 1].parent_i = i;
                                cellDetails[i - 1][j + 1].parent_j = j;
                        }
                }
        }
if (isValid(i - 1, j - 1) == true) {

                if (isDestination(i - 1, j - 1, dest) == true) {

                        cellDetails[i - 1][j - 1].parent_i = i;
                        cellDetails[i - 1][j - 1].parent_j = j;
                        cout<<"\u\u destination cell is found\n";
                        tracePath(cellDetails, dest);
                        foundDest = true;
                        return;
                }


                else if (closedList[i - 1][j - 1] == false
                                && isUnBlocked(grid, i - 1, j - 1)
                                                == true) {
                        gNew = cellDetails[i][j].g + 1.414;
                        hNew = calculateHValue(i - 1, j - 1, dest);
                        fNew = gNew + hNew;

        if (cellDetails[i - 1][j - 1].f == FLT_MAX
                                || cellDetails[i - 1][j - 1].f > fNew) {
```

```
                                openList.insert(make_pair(
                                        fNew, make_pair(i - 1, j - 1)));

                                cellDetails[i - 1][j - 1].f = fNew;
                                cellDetails[i - 1][j - 1].g = gNew;
                                cellDetails[i - 1][j - 1].h = hNew;
                                cellDetails[i - 1][j - 1].parent_i = i;
                                cellDetails[i - 1][j - 1].parent_j = j;
                        }
                }
        }

if (isValid(i + 1, j + 1) == true) {

                if (isDestination(i + 1, j + 1, dest) == true) {

                        cellDetails[i + 1][j + 1].parent_i = i;
                        cellDetails[i + 1][j + 1].parent_j = j;
                        cout<<"\u\udestination\ucell\uis\ufound\n";
                        tracePath(cellDetails, dest);
                        foundDest = true;
                        return;
                }

    else if (closedList[i + 1][j + 1] == false
                                && isUnBlocked(grid, i + 1, j + 1)
                                                == true) {
                        gNew = cellDetails[i][j].g + 1.414;
                        hNew = calculateHValue(i + 1, j + 1, dest);
                        fNew = gNew + hNew;

        if (cellDetails[i + 1][j + 1].f == FLT_MAX
                                || cellDetails[i + 1][j + 1].f > fNew) {
                                openList.insert(make_pair(
                                        fNew, make_pair(i + 1, j + 1)));


                                cellDetails[i + 1][j + 1].f = fNew;
                                cellDetails[i + 1][j + 1].g = gNew;
                                cellDetails[i + 1][j + 1].h = hNew;
                                cellDetails[i + 1][j + 1].parent_i = i;
                                cellDetails[i + 1][j + 1].parent_j = j;
                        }
                }
        }

if (isValid(i + 1, j - 1) == true) {

                if (isDestination(i + 1, j - 1, dest) == true) {

                        cellDetails[i + 1][j - 1].parent_i = i;
                        cellDetails[i + 1][j - 1].parent_j = j;
                        cout<<"\u\udestination\ucell\uis\ufound\n";
                        tracePath(cellDetails, dest);
                        foundDest = true;
                        return;
                }
```

```
            else if (closedList[i + 1][j - 1] == false
                                    && isUnBlocked(grid, i + 1, j - 1)
                                                    == true) {
                                gNew = cellDetails[i][j].g + 1.414;
                                hNew = calculateHValue(i + 1, j - 1, dest);
                                fNew = gNew + hNew;


                if (cellDetails[i + 1][j - 1].f == FLT_MAX
                                    || cellDetails[i + 1][j - 1].f > fNew) {
                                    openList.insert(make_pair(
                                            fNew, make_pair(i + 1, j - 1)));

                                    // Update   details of this cell
                                    cellDetails[i + 1][j - 1].f = fNew;
                                    cellDetails[i + 1][j - 1].g = gNew;
                                    cellDetails[i + 1][j - 1].h = hNew;
                                    cellDetails[i + 1][j - 1].parent_i = i;
                                    cellDetails[i + 1][j - 1].parent_j = j;
                            }
                            }
                }
        }

    if (foundDest == false)
                cout<<"Failed to find   Destination Cell\n";

        return;
}

int main()
{
        /* Description of   Grid-
        1-->   cell is not blocked
        0-->   cell is blocked */
        int grid[ROW][COL]
                = { { 1, 0, 1, 1, 1, 1, 0, 1, 1, 1 },
                        { 1, 1, 1, 0, 1, 1, 1, 0, 1, 1 },
                        { 1, 1, 1, 0, 1, 1, 0, 1, 0, 1 },
                        { 0, 0, 1, 0, 1, 0, 0, 0, 0, 1 },
                        { 1, 1, 1, 0, 1, 1, 1, 0, 1, 0 },
                        { 1, 0, 1, 1, 1, 1, 0, 1, 0, 0 },
                        { 1, 0, 0, 0, 0, 1, 0, 0, 0, 1 },
                        { 1, 0, 1, 1, 1, 1, 0, 1, 1, 1 },
                        { 1, 1, 1, 0, 0, 0, 1, 0, 0, 1 } };


        Pair src = make_pair(8, 0);


        Pair dest = make_pair(0, 0);

        aStarSearch(grid, src, dest);

        return (0);
}
```

5 Complexity Analysis

The time complexity of the A* algorithm is not analytically determinable. As this algorithm takes heuristics into account and heuristics comes in every effective form , it leaves room for different answers.

Therefore it is sensible to find the worst case of complexity of this algorithm. Given a graph with vertices $|V|$ and edges $|E|$, it would be an absolute nightmare for someone to visit each and every vertices via each and every edges. In that case the complexity will be $\mathcal{O}(|V| + |E|)$.

Though, by transforming the graph into a tree, one may make better sense of it. Considering each vertices leaves b numbers of successors or child and the depth of the goal vertex is d. Therefore the time complexity would be $\mathcal{O}(b^d)$. And $\mathcal{O}(b^d)$ would roughly be the space complexity too. Because all the nodes must be to stored in the memory.

6 Use cases of A* search algorithm

A* search algorithm is frequently used to find the shortest path and also in traversing graphs. Even though it was initially designed as an algorithm for graph traversing, it is mostly used by game developers[4] for video games that use path-finding strategies. A* search algorithm uses a heuristic function in deciding the quickest path to be followed for which it is being used in video games. Furthermore, it can detect blocked paths and find the best alternative route to reach its goal. In addition, it also keeps a record of the paths that have been traversed and the paths that are yet to be traversed. So it is efficient in Traffic Navigational Systems as well. However, as it uses a heuristic approach, the environment has to be known entirely first for it to work.

6.1 Practical uses of A* search algorithm

Some of the practical uses of the A* search algorithm has been given below:

- **Artificial Intelligence (AI):** A* search algorithm is often used in AI in computational problems such as path searching problems where the problems might need to be mapped into graphs so that the possible outcomes can be represented by nodes. When the first intelligent robot, Shakey, was invented, A* search algorithm was a part of it as Shakey could plan its actions while recognizing its environment. As A* search algorithm is frequently described as an algorithm that has brains, it stands out among other search algorithms. Moreover, it can find the best path with minimal cost in terms of distance and time as the algorithm is expressed in weighted graphs. Hence, it is being used in AI.
- **Google Map:** A* search algorithm determines the most efficient route and if a path is blocked due to a traffic jam, it finds an alternative path that would take less time. As Google Map needs to find the shortest route to a destination as well as find alternative routes in real-time, A* search algorithm is used. A* search algorithm is formulated in weighted graphs, in this case, this weight refers to the travel time. It starts with a particular node and from there, it creates a tree of paths and expands the paths one at a time until one of its paths ends at the destination which was already determined beforehand by the user.

- **Video Games:** Tower Defense Games are strategic where the goal is to defend the player's territories from enemies by constructing various defensive structures in the path where the enemy might attack, A* search algorithm is used in such games as it finds the shortest path from one node to another. Hence, it can be used for each enemy to search the path leading to the goal. Some of the games that use the A* search algorithm are- Minecraft, Warcraft, etc. It is also used in other genres of video games such as racing games, role-playing games, and turn-based games.

Apart from these, A* search algorithm is used in various other problems that require pathfinding or graph traversing where there is an open list and a closed list.

7 Recommended Use

As the A* search algorithm considers providing the shortest path with minimal cost from the starting node, it can be used diversely in many cases. However, the A* search algorithm is recommended not to use them in complex or large-scale problems as its time and space complexity is very high and not analytic. As it is being used from robotic to AI, possible brand new uses would be earth shaking. But choice of heuristics makes it flexible to implement in a suitable way. Here are few possible recommendations:

- Automated factories and warehouses.
- Automated Bank vault accessing system.
- Particle Physics Experiments, specially in collider.
- Farming field is practically a huge piece of graph, For any optimization problem A* might come handy.

8 Comparison with Similar Algorithms

Due to its completeness, optimality, and efficiency, A* is a graph traversal and path search method that is widely utilized in many fields of computer science. $\mathcal{O}(b^d)$ which is the space complexity of A* algorithm is one of its primary practical drawbacks, as it saves all created nodes in memory. As a result, algorithm outperforms it in practical travel-routing systems. Thus, it is often outperformed in actual travel-routing systems by algorithms that can pre-process the graph to achieve higher performance, as well as memory-bounded techniques; nonetheless, in many circumstances, A* is still the best answer.

Best-first search is a graph-exploration algorithm that expands the most promising node chosen according to a set of rules. An example of a best-first search algorithm is the A* search algorithm. Because they include the distance from the start as well as predicted distances to the objective, neither A* nor B* is a greedy best-first search. In the worst-case scenario, the time and space complexity for the best-case scenario The initial search is the same as with BFS: time is $\mathcal{O}(b^d)$. What distinguishes A* from a greedy best-first search algorithm is that it takes into consideration the cost/distance previously gone, $g(n)$.

A* employs a priority queue, while BFS utilizes a queue. Queues are generally faster than priority queues (for example, Dequeue() is $\mathcal{O}(1)$ versus $\mathcal{O}(\log n)$. A* has the advantage of expanding many fewer nodes than BFS in most cases, but if this isn't the case, BFS will be faster. This can occur if the heuristic used is ineffective and poor, the graph is sparse or small, or the heuristic fails for a specific graph.We should also note that BFS is only helpful for graphs that are not weighted. If the graph is weighted, one needs to employ Dijkstra's Algorithm, BFS's older brother. Because this algorithm employs a priority queue, it should virtually never be quicker than A*, unless the heuristic fails.

DFS (depth-first search) is a tree or graph data structure traversal and search algorithm. The algorithm starts at the root node (in the case of a graph, any random node can be used as the root node) and examines each branch as far as feasible before returning to the root node. DFS's time and space analysis varies depending on the application. DFS is commonly used to traverse a complete graph in theoretical computer science, and it takes time $\mathcal{O}(|V| + |E|)$ where $|V|$ is the number of vertices and $|E|$ is the number of edges. In terms of graph size, this follows a linear pattern. In the worst-case scenario, it saves both the stack of vertices on the current search path and the collection of vertices that have already been visited. Thus, the time and space limitations are the same as for breadth-first search, and the choice of which of these two algorithms to choose is based on the various qualities of the vertex orderings produced by the two algorithms rather than their complexity.If the target is on the first branch, a depth-first search may outperform A* and BFS. There are also some constants to consider. A* stores several states on the OPEN/CLOSED lists, whereas DFS only needs one copy of each state. In certain instances, however, A* should be used instead.

It's worth noting that in a unidirectional search with a consistent heuristic, A* performs the fewest expansions necessary to verify that the answer is optimal. Dijkstra's algorithm is a method for determining the shortest pathways between nodes in a graph, which might be used to depict road networks. There are numerous variations of the algorithm. The original Dijkstra algorithm discovered the shortest path between two given nodes, but a more frequent form fixes a single node as the "source" node and finds shortest pathways from the source to all other nodes in the network, resulting in a shortest-path tree. The time and space complexity of Dijkstra's algorithm is Time: $\mathcal{O}((|V| + |E|)logV)$ and Space: $\mathcal{O}(|V| + |E|)$. Some frequent variants of Dijkstra's algorithm can be thought of as a particular instance of A*, with the heuristic $h(n) = 0$ for all nodes; Dijkstra and A*, in turn, are both special examples of dynamic programming. A* is a specific case of a branch and bound generalization.

A* is essentially a refined version of Dijkstra. A* is referred to as a "best first search" because it chooses which vertex to explore next based on the value of $f(v)$ $[f(v) = h(v) + g(v)]$, where h is the heuristic and g is the current cost. But, if we ever use a non-informative heuristic function: $h(v) = 0$ for each v, A* picks which vertex to develop next based solely on the "so far cost" $g(v)$, which is the same as Dijkstra's algorithm - therefore if $h(v) = 0$, A* falls back to Dijkstra's Algorithm. Hence, we see that A* search has has two cost function.($g(n)$: same as Dijkstra. The real cost to reach a node n. And, $h(n)$: approximate cost from node n to goal node. It is a heuristic function. This heuristic function should never overestimate the cost. That means, the real cost to reach goal node from node n should be greater than or equal h(n). It is called admissible heuristic.). So, for A* The total cost of each node is calculated by $f(n) = g(n) + h(n)$ and for Dijkstra It contains

only one cost function: $f(n) = g$, which is the real cost value from the source to each node (n) It calculates the shortest path from the source to each other node based solely on real costs. Therefore, Dijkstra can never be better in terms of Big-$\mathcal{O}$ run-time complexity as $h(v) = 0$ for all vertices.

To summarize, the A* algorithm, like everything else, has both advantages and disadvantages. Its benefits include the following: It is comprehensive and ideal. It is the most effective approach among the others. It's utilized to solve really difficult tasks. It is the most efficient method, in the sense that no other algorithm is guaranteed to expand fewer nodes than A*. It also has some drawbacks. If the branching factor is finite and every action has a fixed cost, this method is complete. The accuracy of the heuristic approach used to compute h determines how quickly A* search may be completed (n). It has issues with complexity.

9 Conclusion

If an algorithm efficiently solves the problem that has been assigned to it, then it has served its purpose. But A* search algorithm seem more interactive as it uses heuristic function which makes it more reliable than other search algorithms where the environment is completely known.We have found the implementation of the graph very challenging. It would have been more effective to study that If had a little more time. However we intend to do that anyway. Overall it was an thought provoking experience for us.

References

[1] Richard Bellman *Eye of the Hurricane: An Autobiography (1984).*

[2] A* Search Algorithm *https://www.geeksforgeeks.org/a-search-algorithm/*

[3] https://en.wikipedia.org/wiki/Shakey$_t he_r obot$

[4] Abdul Rafiq *et al 2020 IOP Conf. Ser.: Mater. Sci. Eng. 769 012021*

[5] https://www.gatevidyalay.com/tag/a-algorithm-flowchart/

[6] Tabassum, Mujahid. (2013). *Experimental Comparison of Uninformed and Heuristic AI Algorithms for N Puzzle Solution.*

[7] https://www.mygreatlearning.com/blog/a-search-algorithm-in-artificial-intelligence/

[8] https://www.geeksforgeeks.org/a-search-algorithm/

[9] https://en.wikipedia.org/wiki/A*$_s earch_a lgorithm Applications$

[10] https://ramprakasharava.wordpress.com/2016/06/18/a-search-algorithm/?fbclid=IwAR3T62Qupfg-ytdsmdoIKiIlatGs4QV$_Q QyTu_w tAKEGGhfndkbbJaVZAi$0

[11] https://www.quora.com/How-does-the-algorithm-of-Google-Maps-work

[12] https://www.javatpoint.com/ai-informed-search-algorithms